

Beauty Marketplace - Entrega 4 - Padrões, APIs e IA

1. Identificação do grupo

- Número do grupo: [preencher número do grupo]
- Integrante 1: [nome completo] - RA [RA]
- Integrante 2: [nome completo] - RA [RA]
- Integrante 3: [nome completo] - RA [RA]
- Integrante 4: [nome completo] - RA [RA]

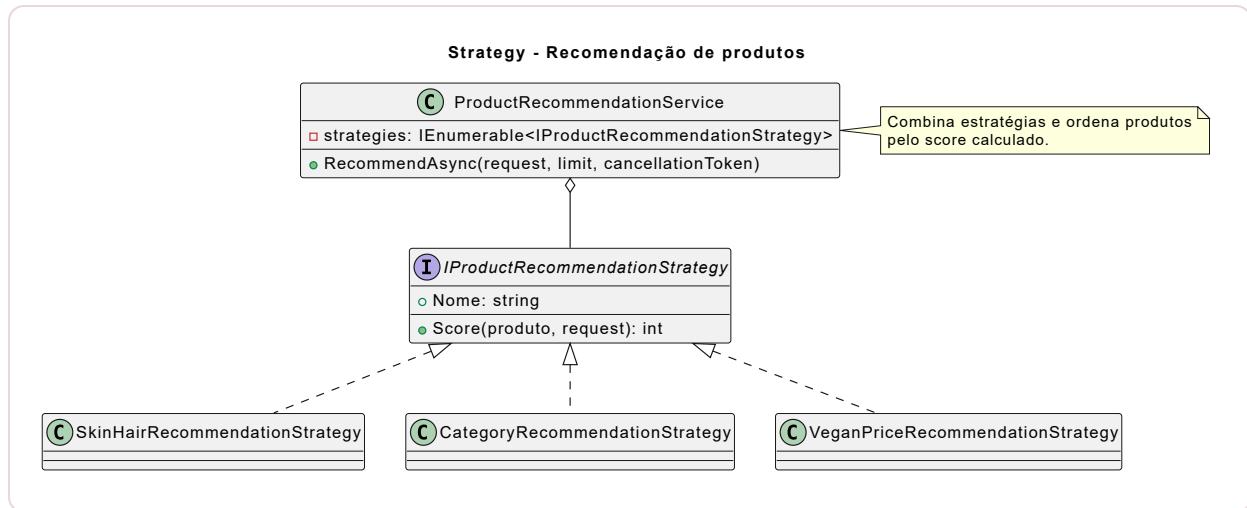
Objetivo

Apresentar os padrões de projeto aplicados no Beauty Marketplace, a documentação das APIs do sistema e a prova de conceito de Inteligência Artificial para recomendação personalizada de produtos de beleza.

2. Padrões de projeto GoF

2.1 Strategy - Recomendação de produtos

- **Categoria:** Comportamental.
- **Problema resolvido:** a recomendação precisa combinar critérios diferentes, como tipo de pele, tipo de cabelo, categoria, produto vegano e preço, sem deixar todo o cálculo concentrado em um único método.
- **Aplicação no projeto:** `IProductRecommendationStrategy`, `SkinHairRecommendationStrategy`, `CategoryRecommendationStrategy`, `VeganPriceRecommendationStrategy` e `ProductRecommendationService`.
- **Diagrama UML:** `padroes/uml/strategy-recomendacao.puml`.



Trecho de código:

```

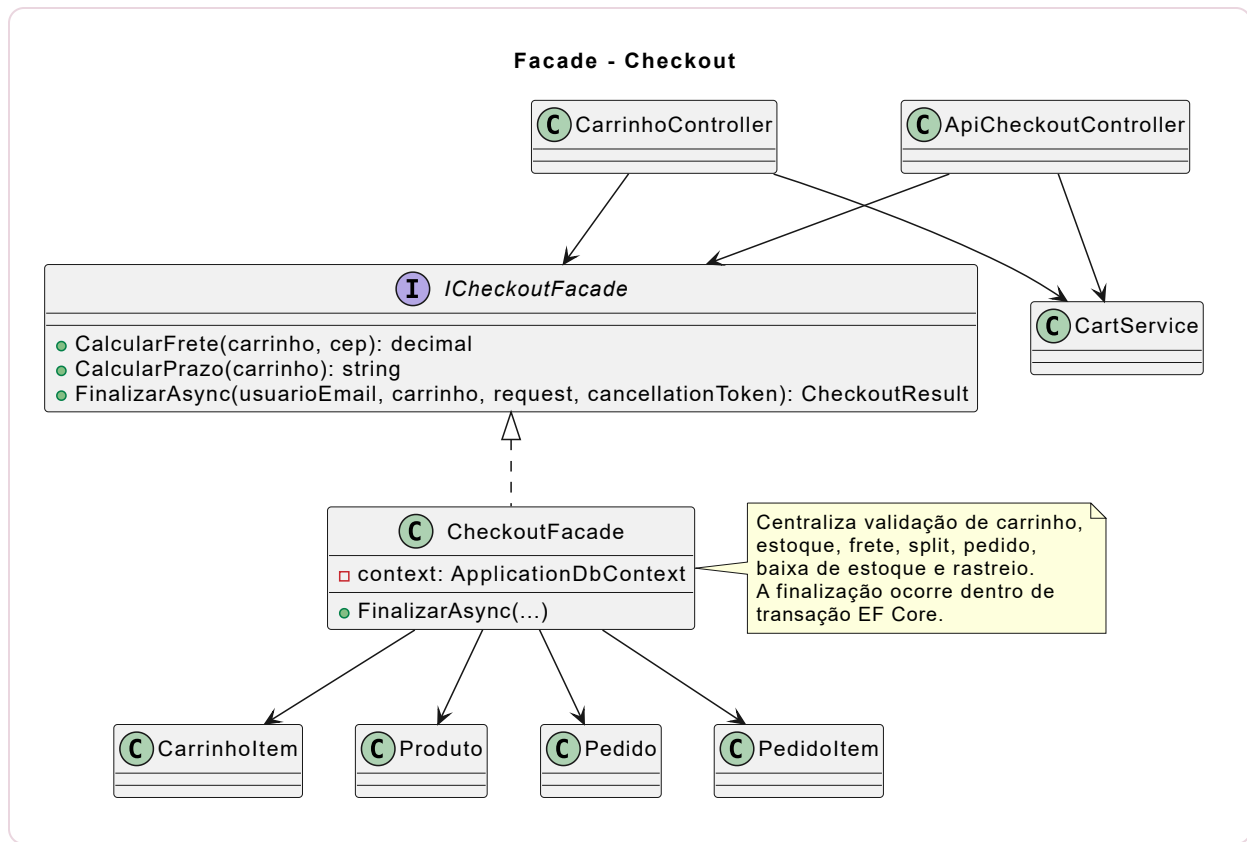
public interface IProductRecommendationStrategy
{
    string Nome { get; }
    int Score(Produto produto, ProductRecommendationRequest request);
}

public sealed class ProductRecommendationService
{
    private readonly IEnumerable<IProductRecommendationStrategy> strategies;

    public Task<IReadOnlyList<Produto>> RecommendAsync(ProductRecommendationRequest request, int limit, CancellationToken ct)
    {
        // Cada Strategy calcula parte do score de recomendação
    }
}
  
```

2.2 Facade - Checkout

- **Categoria:** Estrutural.
- **Problema resolvido:** finalizar uma compra envolve validar carrinho, conferir estoque, calcular frete, gerar split, criar pedido, criar itens e baixar estoque. Sem uma fachada, essa regra ficaria espalhada pelos controllers.
- **Aplicação no projeto:** `ICheckoutFacade` e `CheckoutFacade`, usados pelo endpoint `POST /api/checkout` e pelo checkout MVC. A finalização roda dentro de transação EF Core, validando estoque, criando pedido, baixando estoque e liberando a limpeza do carrinho somente após sucesso.
- **Diagrama UML:** `padroes/uml/facade-checkout.puml`.



Trecho de código:

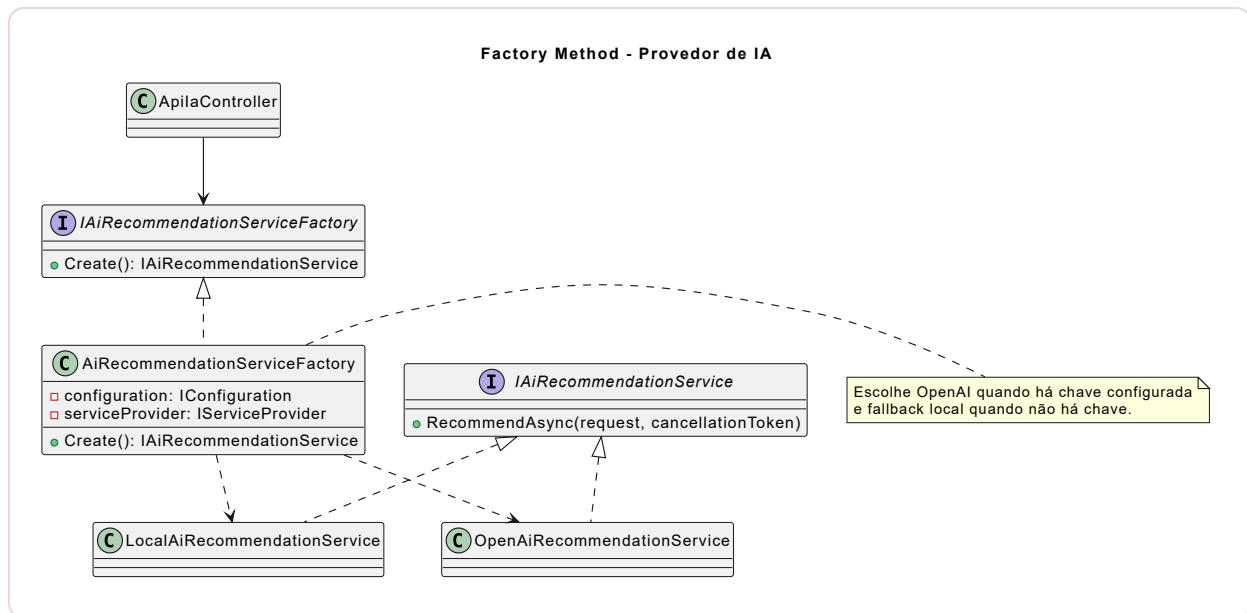
```

public interface ICheckoutFacade
{
    decimal CalcularFrete(IReadOnlyCollection<CarrinhoItem> carrinho, string? cep);
    string CalcularPrazo(IReadOnlyCollection<CarrinhoItem> carrinho);
    Task<CheckoutResult> FinalizarAsync(string usuarioEmail,
    IReadOnlyCollection<CarrinhoItem> carrinho, CheckoutRequest request, CancellationToken
    ct);
}

```

2.3 Factory Method - Provedor de IA

- **Categoria:** Criacional.
- **Problema resolvido:** o sistema precisa usar OpenAI quando houver chave configurada, mas também funcionar em apresentação acadêmica sem credenciais externas.
- **Aplicação no projeto:** `IAiRecommendationServiceFactory`, `AiRecommendationServiceFactory`, `OpenAiRecommendationService` e `LocalAiRecommendationService`.
- **Diagrama UML:** `padroes/uml/factory-method-ia.puml`.



Trecho de código:

```

public sealed class AiRecommendationServiceFactory : IAIRecommendationServiceFactory
{
    public IAIRecommendationService Create()
    {
        var apiKey = _configuration["OpenAI:ApiKey"] ??
            Environment.GetEnvironmentVariable("OPENAI_API_KEY");

        return string.IsNullOrEmpty(apiKey)
            ? _serviceProvider.GetRequiredService<LocalAiRecommendationService>()
            : _serviceProvider.GetRequiredService<OpenAiRecommendationService>();
    }
}

```

3. Documentação de APIs

O projeto expõe documentação Swagger em:

- Swagger UI local: <http://localhost:5016/swagger>
- OpenAPI JSON local: <http://localhost:5016/swagger/v1/swagger.json>
- Arquivo versionado: `api/openapi.json`
- Postman Collection: `api/postman_collection.json`

O Swagger/OpenAPI final foi filtrado para documentar somente endpoints cujo caminho começa com `/api`. As rotas MVC usadas pela interface web, como ações internas do carrinho, não entram na documentação oficial da Entrega 4. A especificação também inclui um esquema informativo de autenticação por cookie do ASP.NET Identity, mantendo o mecanismo atual de login do site.

Os endpoints de carrinho, checkout e pedidos usam autenticação por cookie do ASP.NET Identity e exigem usuário com role `Consumidor`. Quando acessados sem login, retornam `401`.

Unauthorized ; quando acessados por perfil sem permissão, retornam 403 Forbidden .

Endpoints documentados

Endpoint	Método	Descrição	Status principais
/api/produtos	GET	Lista produtos com filtros de beleza.	200
/api/produtos/{id}	GET	Consulta detalhes de produto.	200, 404
/api/produtos/{id}/avaliacoes	GET	Lista avaliações do produto.	200, 404
/api/produtos/{id}/recomendacoes	GET	Lista produtos recomendados.	200, 404
/api/carrinho	GET	Retorna carrinho do consumidor.	200, 401
/api/carrinho/itens	POST	Adiciona item ao carrinho.	200, 400, 404
/api/carrinho/itens/{produtoId}	PUT	Atualiza quantidade do item.	200, 404
/api/carrinho/itens/{produtoId}	DELETE	Remove item do carrinho.	200
/api/checkout	POST	Finaliza compra do carrinho.	201, 400, 401
/api/pedidos	GET	Lista pedidos do consumidor.	200, 401
/api/pedidos/{id}	GET	Consulta pedido com rastreio.	200, 404
/api/ia/recomendacoes	POST	Executa PoC de recomendação por IA.	200, 400

Total documentado: 12 operações /api , acima do mínimo de 10 endpoints solicitado.

Exemplo de request para IA:

```
{
  "tipoPele": "Oleosa",
  "tipoCabelo": "Cacheado",
  "objetivo": "hidratar",
  "vegano": true
}
```

```
"precoMax": 120
```

Exemplo de response:

```
{
  "provedor": "Local demonstrativo",
  "resumo": "Recomendação gerada por regras locais que simulam a camada de IA para apresentação sem chave externa.",
  "alertaCompatibilidade": "As sugestões não substituem avaliação dermatológica ou profissional.",
  "produtos": [
    {
      "produtoId": 1,
      "nome": "Base Líquida Maybelline NY Fit Me Matte FPS22",
      "marca": "Maybelline",
      "categoria": "Maquiagem",
      "preco": 51.30,
      "imagemUrl": "/images/produtos/ reais/base-liquida-maybelline-ny-fit-me-matte-fps22.jpg",
      "detalhesUrl": "/Produto/Detalhes/1",
      "motivo": "Combina com pele oleosa e acabamento matte."
    }
  ]
}
```

4. Inteligência Artificial na aplicação

A IA será usada para recomendação personalizada de produtos de beleza. O consumidor informa tipo de pele, tipo de cabelo, objetivo, preferência vegana e faixa de preço. A aplicação retorna produtos compatíveis com justificativa, imagem e link de detalhes, permitindo renderizar cards diretamente na interface.

Ferramenta escolhida

- **OpenAI Responses API:** escolhida por permitir geração de texto e respostas estruturadas em uma API atual para aplicações com IA.
- **Fallback local:** usado quando não há chave `OPENAI_API_KEY`, garantindo que a PoC funcione durante apresentação.

Referências oficiais:

- OpenAI Platform: <https://platform.openai.com/docs>
- Responses API: <https://platform.openai.com/docs/api-reference/responses>
- Structured Outputs: <https://platform.openai.com/docs/guides/structured-outputs>

PoC implementada

Endpoint: `POST /api/ia/recomendacoes`

Além do endpoint JSON, o projeto possui uma tela para demonstração em **Consumidor > Recomendação IA**. Assim, durante a apresentação, o consumidor consegue testar a PoC sem depender do Postman e visualizar cards com imagem, marca, preço, motivo da recomendação, botão de detalhes e botão de carrinho.

Fluxo:

1. O endpoint recebe o perfil de beleza do consumidor.
2. A fábrica escolhe OpenAI quando existe chave configurada.
3. Sem chave, o fallback local usa as estratégias de recomendação.
4. A resposta retorna produtos aprovados, justificativa, imagem, link de detalhes, provedor usado e alerta de compatibilidade.

5. Checkpoint 2 - Estado atual do projeto

Concluído

- Marketplace com separação de perfis: consumidor, lojista e administrador.
- Catálogo com filtros de beleza, slug único, 60 produtos seedados por JSON, imagens reais locais e curadoria de preços compatível com o mercado brasileiro.
- Painel do lojista com cadastro/edição de produtos e upload validado de imagem.
- Carrinho multi-lojista com persistência por 7 dias, sincronizando sessão e banco local.
- Checkout transacional via Facade, com split, frete, rastreo e baixa de estoque.
- Lista de desejos, avaliações, pedidos e dashboard do lojista.
- Lojista pode atualizar status de envio dos próprios itens de pedido.
- Área administrativa para aprovação de lojistas, moderação de produtos e comissões.
- Catálogo, API e IA exibem apenas produtos aprovados.
- Entrega 3 organizada com C4, SQL, MongoDB e Redis.
- APIs JSON com Swagger/OpenAPI filtrado para `/api`.
- PoC de IA para recomendação.
- Página de Recomendação IA para demonstração pelo perfil consumidor.
- Testes automatizados cobrindo Strategy, Facade/checkout, carrinho persistente, moderação, IA e proteção por perfil.

Em andamento

- Evolução de integrações reais de pagamento, frete, e-mail e IA externa.
- Testes manuais finais para apresentação.

Links

- Repositório GitHub:
<https://github.com/EduardoSilvaNegreiros/TrabalhoFaculdade5Semestre2026>
- GitHub Projects/Kanban: <https://github.com/users/EduardoSilvaNegreiros/projects/2>
- Nome do Kanban: **Kanban - Projeto 5 Semestre ADS**
- Swagger local: <http://localhost:5016/swagger>
- Postman Collection: docs/Entrega 4/api/postman_collection.json

Validação técnica executada

- `dotnet restore` : restauração concluída.
- `dotnet build --no-restore` : compilação concluída com **0 erros e 0 warnings**.
- `dotnet test --no-build` : **15 testes aprovados**, cobrindo recomendação, checkout, carrinho persistente, roles, catálogo, imagens locais, upload, moderação, IA com cards e proteção por lojista.
- `dotnet list package --vulnerable --include-transitive` : conferido sem vulnerabilidades conhecidas.

Conferência de commits

Conferência realizada em 31/05/2026. O repositório público e o Kanban foram conferidos. O histórico local visível mostra commits associados a:

- Eduardo <edunegreiros@gmail.com>
- Eduardo Silva de Negreiros <edunegreiros@gmail.com>

Observação: caso o professor exija evidência individual de todos os integrantes, os demais membros devem realizar commits no repositório ou serem incluídos como coautores nos commits.

Critérios de avaliação

Critério	Atendimento
Padrões GoF	Strategy, Facade e Factory Method aplicados e documentados.
Documentação de APIs	Swagger/OpenAPI e Postman com 12 operações <code>/api</code> , métodos HTTP, parâmetros, exemplos e status codes.
Inteligência Artificial	PoC de recomendação com OpenAI planejado, fallback local funcional e tela com cards de produto.
Checkpoint 2	Estado atual, pendências, GitHub, Kanban e commits conferidos.